Adapter Pattern

Problem → Alternatives & SOLID trade-offs → Adapter Intuition → Detailed Java Example

---

1. Problem Statement

The concrete engineering problem we are solving

We have a ranking service that needs to rank sellers for a given SKU using a "seller search" service. Initially, the system uses `SDSellerSearchService` (Snapdeal) with methods like:

- `getSellersBySKU(String sku) : List<SDVendor>`
- `getSellerwithMaxDiscount(String sku) : SDVendor`

Later, Snapdeal acquires **Exclusively**, which provides a similar capability via `ExclusivelySellerSearchService` but with *different method signatures, data types, and naming* (e.g., pagination, different price units, scores instead of ratings).

Our `SellerRankingService` contains ranking algorithms and *should not be polluted* with the shape or quirks of either provider. We want the ranking to work with **either** service (and future ones) by *swapping an abstraction* rather than rewriting the ranking code.

---

2. Constraints, Goals, and Non-Goals

- **We cannot freely modify legacy classes/services**. They are owned by other teams, have other callers, or are third-party.
- **The ranking service must depend on a stable interface**, not on the concrete Snapdeal or Exclusively services.
- **New providers should be pluggable** with minimal change to existing code (*Open/-Closed*).
- **Mapping differences** (IDs, fields, units, ranges) must live in a *small, testable boundary*, not spread across the codebase.
- **Constructor/DI-friendly**: The ranking service should receive an abstraction via dependency injection.

---

3. Naïve Alternatives and Why They Fail (Through SOLID)

3.1 Change the Legacy Services to Match Our Needs

Question: Why not just alter `SDSellerSearchService` and `ExclusivelySellerSearchService` to expose a single, uniform API?

Answer: This often violates:

- **Open/Closed Principle (OCP)**: Legacy services should be *open for extension, closed for modification.* Changing them risks breaking other consumers and triggering broad retesting.
- **Single Responsibility Principle (SRP)**: Their responsibility is to serve their domain. Forcing them to cater to a new canonical interface couples them to our downstream client.
- **Dependency Inversion (DIP)**: It inverts the desired direction; instead of our high-level ranking module depending on an abstraction, the low-level modules get twisted to our needs.

Practically, political and organizational constraints also make this hard.

3.2 Put Big `if/else` or `switch` Logic in `SellerRankingService`

Question: Can the ranking service detect the provider and branch?

Answer: This pollutes the ranking code with provider specifics and violates:

- **SRP**: Ranking should rank; it should not parse, map, and normalize vendor data.
- **OCP**: Adding a new provider requires modifying the ranking service again.
- **DIP**: The high-level module depends on low-level details.

You also get *shotgun surgery*: changes ripple to many places.

3.3 A Global "God" Mapper Utility

Question: What if all conversions live in one giant `Mapper`?

Answer: Better than scattering logic, but:

- Internally it's still a *switchyard.* OCP is weak; every new provider reopens the mapper.
- Testability suffers because it centralizes many unrelated conversions.
- Callers still know too much about which provider they're mapping.

3.4 Share a Single Canonical Domain Model Everywhere

Question: Can we make all services use a new shared `Seller` class and push it into both codebases?

Answer: This often violates team boundaries and introduces backward-compat constraints. It also risks **Liskov Substitution (LSP)** issues if semantics differ (e.g., "score" $\neq$ "rating").

3.5 Static Convenience Methods in the Client

Question: What about static helpers directly called by the ranking code?

Answer: This roughly equals option 3; you still couple to concrete providers and lose polymorphism. It also hurts **testability** and **DIP**.

---

4. The Adapter Pattern: The Right Tool

Question: What does the Adapter give us here?

Answer: A **stable target interface** for the client and one *small* **object adapter** for each legacy provider that translates shapes/semantics *at the boundary.* This satisfies:

- **SRP**: Ranking ranks; adapters adapt.
- **OCP**: New providers → new adapters. Ranking untouched.
- **LSP**: Adapters ensure consistent semantics to the target interface.
- **ISP**: The target interface is minimal and client-oriented.
- **DIP**: `SellerRankingService` depends on an abstraction (`SellerSearch`), not concrete services.

4.1 Intuition

A travel plug adapter: one side matches the wall socket (legacy provider), the other matches your laptop plug (client's interface). Neither wall nor laptop changes; only the adapter speaks both dialects.

4.2 Object vs. Class Adapter (Java)

Question: Which variant should we use?

Answer: **Object adapter** (composition) is standard in Java. Class adapters require multiple inheritance semantics (not available for classes in Java) and couple you tightly to the adaptee.

---

5. Building the Abstraction First

Question: What should the *client-oriented* target interface look like?

Answer: Define a minimal API the ranking needs and a canonical `Seller` shape:

Listing 1: Target interface and canonical model

```
/* SellerSearch.java */
import java.util.*;

public interface SellerSearch {
  List<Seller> getSellersBySku(String sku);
  Optional<Seller> getSellerWithMaxDiscount(String sku);
}

/* Seller.java */
public final class Seller {
  public final String id;
  public final String name;
  public final double price;        // major units
  public final double discountPct;  // 0..100
  public final double rating;       // 0..5
  public Seller(String id, String name, double price, double discountPct,
      double rating) {
    this.id = id; this.name = name; this.price = price;
```

```
    this.discountPct = discountPct; this.rating = rating;
  }
  @Override public String toString() {
    return String.format("%s(%s) price=%.2f, discount=%.1f%%, rating=%.1f/5
        ",
      name, id, price, discountPct, rating);
  }
}
```

---

6. Legacy Providers (Unmodified)

Question: What are we adapting?

Answer: Snapdeal and Exclusively services, each with their own signatures and data models.

6.1 Snapdeal (Adaptee A)

Listing 2: Snapdeal service and DTOs

```
/* SDVendor.java */
public final class SDVendor {
  public final String vendorId;
  public final String shopName;
  public final double listPrice;
  public final double discountPct;  // 0..100
  public final double starRating;   // 0..5
  public SDVendor(String id, String shop, double price, double discount,
      double stars) {
    this.vendorId = id; this.shopName = shop; this.listPrice = price;
    this.discountPct = discount; this.starRating = stars;
  }
}

/* SDSellerSearchService.java */
import java.util.*;

public class SDSellerSearchService {
  public List<SDVendor> getSellersBySKU(String sku) {
    return List.of(
      new SDVendor("S-101", "Trendy Threads", 999.0, 25.0, 4.6),
      new SDVendor("S-102", "Gizmo Galaxy", 1099.0, 15.0, 4.2)
    );
  }
  public SDVendor getSellerwithMaxDiscount(String sku) {
    return getSellersBySKU(sku).stream()
      .max(Comparator.comparingDouble(v -> v.discountPct))
      .orElse(null);
  }
}
```

6.2 Exclusively (Adaptee B)

Listing 3: Exclusively service and DTOs (different signatures)

```java
/* ExMerchant.java */
public final class ExMerchant {
  public final String id;
  public final String display;   // seller name
  public final long pricePaise;  // minor units
  public final int off;          // 0..100
  public final int score100;     // 0..100 quality score
  public ExMerchant(String id, String display, long pricePaise, int off,
      int score100) {
    this.id = id; this.display = display; this.pricePaise = pricePaise;
    this.off = off; this.score100 = score100;
  }
}

/* Page.java */
import java.util.*;
public final class Page<T> {
  public final List<T> data; public final int page; public final int
      perPage; public final long total;
  public Page(List<T> data, int page, int perPage, long total) {
    this.data = data; this.page = page; this.perPage = perPage; this.total
        = total;
  }
}

/* ExclusivelySellerSearchService.java */
import java.util.*;

public class ExclusivelySellerSearchService {
  public Page<ExMerchant> merchantsFor(String articleCode, int page, int
      perPage) {
    List<ExMerchant> all = List.of(
      new ExMerchant("E-201", "Exclusive Couture", 99900, 30, 92),
      new ExMerchant("E-202", "ElectroHub",       99500, 10, 84)
    );
    return new Page<>(all, 1, all.size(), all.size());
  }
  public Optional<ExMerchant> maxDiscounted(String articleCode) {
    return merchantsFor(articleCode, 1, 50).data.stream()
      .max(Comparator.comparingInt(m -> m.off));
  }
}
```

---

7. Adapters (Object Adapters via Composition)

Question: Where do we keep all shape/semantics translation?

Answer: In small adapter classes that implement `SellerSearch` and contain the mapping logic. This localizes risk and is easy to test.

7.1 Snapdeal → SellerSearch

Listing 4: SDSearchAdapter.java

```java
import java.util.*;
```

```java
public final class SDSearchAdapter implements SellerSearch {
  private final SDSellerSearchService sd;
  public SDSearchAdapter(SDSellerSearchService sd) { this.sd = sd; }

  @Override public List<Seller> getSellersBySku(String sku) {
    List<SDVendor> vendors = sd.getSellersBySKU(sku);
    List<Seller> out = new ArrayList<>();
    for (SDVendor v : vendors) {
      out.add(new Seller(
        v.vendorId, v.shopName,
        v.listPrice, v.discountPct, v.starRating
      ));
    }
    return out;
  }

  @Override public Optional<Seller> getSellerWithMaxDiscount(String sku) {
    SDVendor v = sd.getSellerwithMaxDiscount(sku);
    return Optional.ofNullable(v).map(x ->
      new Seller(x.vendorId, x.shopName, x.listPrice, x.discountPct, x.
          starRating)
    );
  }
}
```

7.2 Exclusively → SellerSearch (unit conversions)

Listing 5: ExSearchAdapter.java

```java
import java.util.*;

public final class ExSearchAdapter implements SellerSearch {
  private final ExclusivelySellerSearchService ex;
  public ExSearchAdapter(ExclusivelySellerSearchService ex) { this.ex = ex;
      }

  @Override public List<Seller> getSellersBySku(String sku) {
    Page<ExMerchant> page = ex.merchantsFor(sku, 1, 50);
    List<Seller> out = new ArrayList<>();
    for (ExMerchant m : page.data) {
      double price = m.pricePaise / 100.0;  // paise -> rupees
      double rating = m.score100 / 20.0;    // 0..100 -> 0..5
      out.add(new Seller(m.id, m.display, price, m.off, rating));
    }
    return out;
  }

  @Override public Optional<Seller> getSellerWithMaxDiscount(String sku) {
    return ex.maxDiscounted(sku).map(m ->
      new Seller(m.id, m.display, m.pricePaise / 100.0, m.off, m.score100 /
          20.0)
    );
  }
}
```

8. Client: Constructor Injection of the Abstraction

Question: How do we keep the ranking code provider-agnostic?

Answer: **Inject** a `SellerSearch` into `SellerRankingService`. Swap adapters at composition time.

Listing 6: SellerRankingService.java

```
import java.util.*;

public final class SellerRankingService {
  private final SellerSearch search;
  public SellerRankingService(SellerSearch search) { this.search = search;
      }

  // Example heuristic: discount desc, rating desc, price asc
  public List<Seller> rankBySku(String sku) {
    List<Seller> sellers = new ArrayList<>(search.getSellersBySku(sku));
    sellers.sort(Comparator
      .comparingDouble((Seller s) -> s.discountPct).reversed()
      .thenComparingDouble(s -> s.rating).reversed()
      .thenComparingDouble(s -> s.price));
    return sellers;
  }

  public Optional<Seller> bestDiscount(String sku) {
    return search.getSellerWithMaxDiscount(sku);
  }
}
```

Listing 7: Demo wiring: Snapdeal vs Exclusively

```
public class Demo {
  public static void main(String[] args) {
    SellerRankingService rankingSD =
      new SellerRankingService(new SDSearchAdapter(new
          SDSellerSearchService()));
    System.out.println("SD ranked: " + rankingSD.rankBySku("SKU-123"));
    System.out.println("SD best : " + rankingSD.bestDiscount("SKU-123"));

    SellerRankingService rankingEX =
      new SellerRankingService(new ExSearchAdapter(new
          ExclusivelySellerSearchService()));
    System.out.println("EX ranked: " + rankingEX.rankBySku("SKU-123"));
    System.out.println("EX best : " + rankingEX.bestDiscount("SKU-123"));
  }
}
```

---

9. Practical Concerns: Mapping, Errors, Performance

Question: What should adapters take care of beyond "happy path" mapping?

Answer:

- **Units and Ranges**: Minor ↔ major units (paise/rupees), 0..100 scores ↔ 0..5 ratings.
- **Nullability** and **missing fields**: Use safe defaults or throw domain-specific exceptions.

- **Unknown enums / tags**: Map to `UNKNOWN` and log.
- **Error translation**: Convert provider-specific exceptions to client-facing error types.
- **Caching / batching**: Adapters are a good place to add read-through cache or batch calls *without* leaking provider details to the client.
- **Observability**: Log at the boundary; attach request IDs/metrics here.

---

10. Why Adapter Honors SOLID (Explicitly)

Question: Map this design to SOLID.
Answer:

- **SRP**: Ranking ranks. Each adapter adapts one provider. No class juggles multiple responsibilities.
- **OCP**: Adding a provider is adding a class (an adapter), not editing `SellerRankingService`.
- **LSP**: All `SellerSearch` implementations provide equivalent semantics to the client.
- **ISP**: `SellerSearch` exposes only what ranking needs; no fat interfaces.
- **DIP**: The high-level module (`SellerRankingService`) depends on the abstraction `SellerSearch`, not on `SDSellerSearchService`/`ExclusivelySellerSearchService`.

---

11. When Not to Use Adapter

Question: When is Adapter the wrong tool?
Answer:

- If you *own both sides* and can simply change one API, do that and delete the adapter.
- If models must truly *merge*, plan a domain convergence; use adapters as a temporary *Anti-Corruption Layer* during migration.
- If the client needs to orchestrate complex provider workflows, consider a **Facade** with adapters behind it.

---

12. Appendix: Employee CSV/DB/LDAP Example (Brief)

Question: How does Adapter look in a simpler data-normalization case?
Answer: Define a target `Employee` interface with `getId/getFirstName/getLastName/getEmail`. Implement `EmployeeCSVAdapter`, `EmployeeDBAdapter`, and `EmployeeLDAPAdapter` to translate from their legacy shapes. The client keeps a `List<Employee>` and prints a unified view. All provider specifics stay in adapters; client logic is pure.

---

13. Takeaway

Adapter lets your code speak one clean language while understanding many—by translating each foreign API at the boundary, keeping the core closed to change and open to extension.